



UNIVERSIDAD DEL VALLE

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

LG14 – INVESTIGACIÓN Y DESPLIEGUE DE UN REPOSITORIO BIG DATA CON DOCKER

ESTUDIANTES:	Edson Marcelo Cayo Ali Joel Julio Salazar Ferrufino Alex Bryam Barrera Alarcon
MATERIA:	Tecnologías Emergentes I
GRUPO:	B
CÓDIGO DE PRÁCTICA	LG14
DOCENTE:	Ing. Efrain Luna Mamani

Cochabamba - Bolivia

Gestión 2 – 2026

1. Información que debe analizarse

1.1 Información general

Información	Detalle
Nombre	Hadoop Sandbox
Autor u organización	hadoop-sandbox
URL	https://github.com/hadoop-sandbox/hadoop-sandbox
Fecha de creación	24 de octubre de 2021
Última actualización	1 de marzo de 2026
Número de estrellas	23
Número de forks	5
Licencia	No se declara una licencia propia en el repositorio
Objetivo del proyecto	Proporcionar un clúster funcional de Apache Hadoop YARN desplegable mediante Docker Compose, incluyendo HDFS, procesamiento distribuido, Spark, monitoreo y acceso mediante un nodo cliente.

1.2 Tecnologías

Tecnología	Análisis
Tecnología Big Data principal	Apache Hadoop: HDFS, YARN y MapReduce
Versión utilizada	Las imágenes actuales se construyen con Hadoop 3.5.0. El Compose utiliza etiquetas latest, por lo que la versión puede cambiar al actualizar las imágenes.
Docker	Sí
Docker Compose	Sí, requiere principalmente Docker Compose 2.x
Sistema operativo base	Ubuntu Noble (24.04) en las imágenes Hadoop
Bases de datos	No utiliza una base de datos relacional o NoSQL. El almacenamiento principal es HDFS.
Frameworks / herramientas adicionales	Apache Spark, Prometheus JMX Exporter, Apache HTTP Server y OpenSSH
Lenguajes utilizados	Principalmente Shell y archivos de configuración YAML/XML. Hadoop y Spark aportan tecnologías basadas principalmente en Java/Scala, pero su código fuente no forma parte de este repositorio.

2. Análisis de la arquitectura

El despliegue predeterminado está compuesto por 13 servicios Docker. Hadoop se divide en servicios especializados para HDFS, YARN y MapReduce, y se agregan Spark, monitoreo JMX, un nodo cliente y un servidor frontal.

Contenedor / servicio	Función	Imagen principal	Puerto / acceso	Dependencia principal
namenode	Administra metadatos de HDFS	hadoop-hdfs-namenode:latest	9870	
namenode-jmx-exporter	Exporta métricas del NameNode	prometheus-jmx-exporter:latest	1128 interno	namenode
datanode	Almacena bloques de datos HDFS	hadoop-hdfs-datanode:latest	9864	namenode
datanode-jmx-exporter	Métricas del DataNode	prometheus-jmx-exporter:latest	1129 interno	datanode
resourcemanager	Administra recursos y aplicaciones YARN	hadoop-yarn-resourcemanager:latest	8088	namenode
resourcemanager-jmx-exporter	Métricas del ResourceManager	prometheus-jmx-exporter:latest	1126 interno	resourcemanager
nodemanager	Ejecuta tareas asignadas por YARN	hadoop-yarn-nodemanager-spark:latest	8042	namenode, datanode y resourcemanager
nodemanager-jmx-exporter	Métricas del NodeManager	prometheus-jmx-exporter:latest	1127 interno	nodemanager
jobhistoryserver	Historial de trabajos MapReduce	hadoop-mapred-jobhistoryserver:latest	19888	namenode, datanode y resourcemanager
sparkhistoryserver	Historial de trabajos Spark	spark-historyserver:latest	18080	namenode, datanode y resourcemanager
jobhistoryserver-jmx-exporter	Métricas del JobHistoryServer	prometheus-jmx-exporter:latest	1130 interno	jobhistoryserver
clientnode	Nodo cliente para utilizar Hadoop y Spark por SSH	hadoop-client-spark:latest	2222 → 22	Servicios principales Hadoop
front	Punto frontal para acceder a las interfaces web	httpd:2.4	8042, 8080, 8088, 9864, 9870, 19888 y 18080	Servicios Hadoop y Spark

El front centraliza el acceso desde la máquina host a las diferentes interfaces web, mientras que clientnode permite acceder mediante SSH.

Redes

El proyecto define dos redes Docker:

- default y front.

La red default permite la comunicación interna del clúster, mientras que front es utilizada por clientnode y front. Algunos contenedores de monitoreo comparten directamente la red del servicio que supervisan mediante network_mode.

Volúmenes

Se definen cinco volúmenes Docker:

Volumen	Uso principal
namenode	Datos persistentes del NameNode
hadoopnode	Datos del nodo Hadoop/DataNode
dnssocket	Comunicación compartida entre DataNode y NodeManager
clientnodehome	Directorio personal del nodo cliente
clientnodessh	Configuración y claves SSH del nodo cliente

Variables de entorno

El despliegue permite modificar mediante .env las imágenes de los diferentes servicios y parámetros como:

- LISTEN_ADDRESS
- LISTEN_HOST

Esto permite cambiar imágenes o la dirección donde se publican los servicios sin editar directamente el Compose.

Entre las variables principales se encuentran:

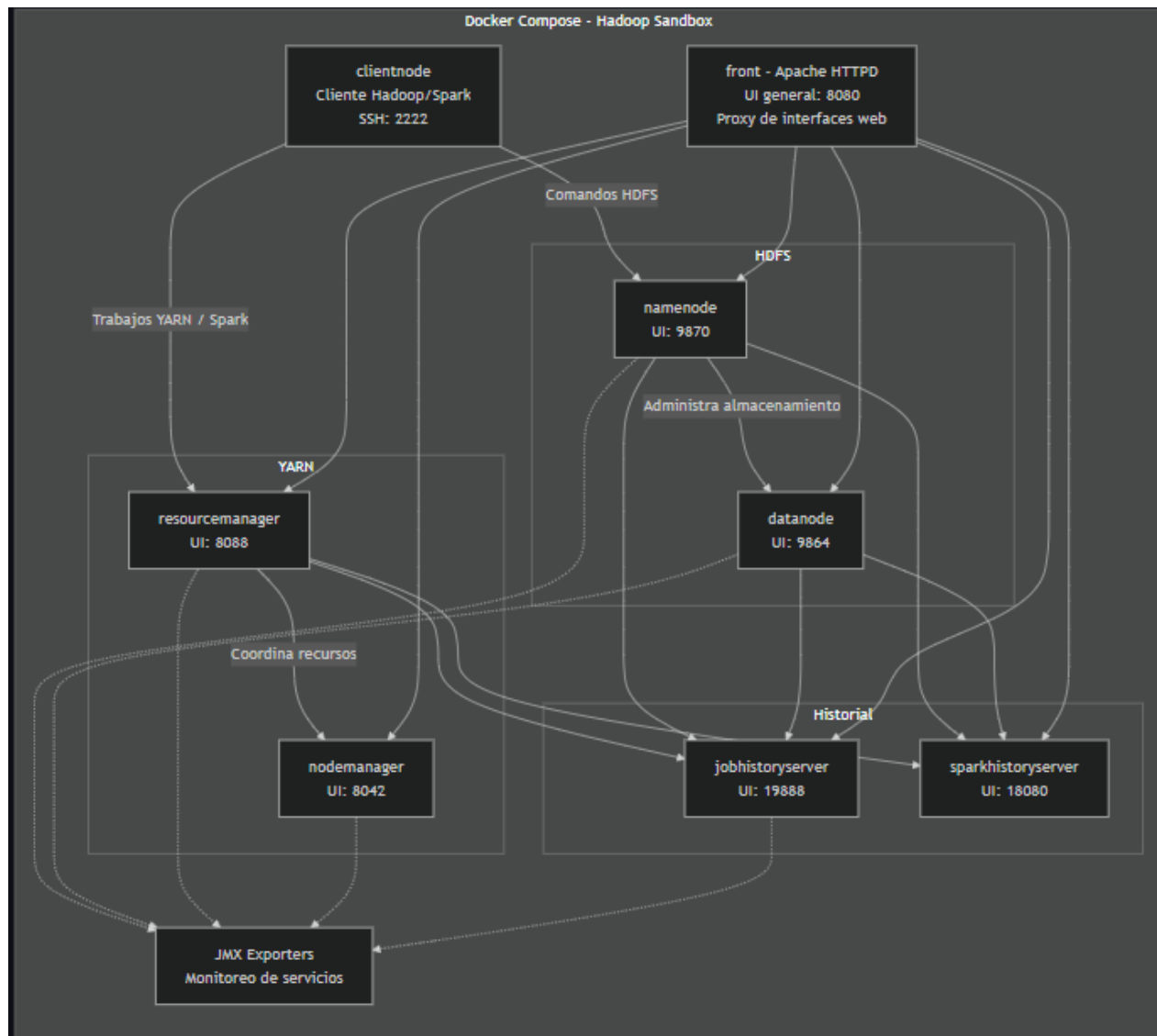
- NAMENODE_IMAGE
- DATANODE_IMAGE
- RESOURCEMANAGER_IMAGE
- NODEMANAGER_IMAGE
- JOBHISTORYSERVER_IMAGE
- SPARKHISTORYSERVER_IMAGE
- CLIENT_IMAGE, JMX_EXPORTER_IMAGE
- HTTPD_IMAGE
- LISTEN_ADDRESS
- LISTEN_HOST

Archivos de configuración

El archivo principal es docker-compose.yaml. También se utilizan configuraciones específicas dentro de conf/, entre ellas archivos Hadoop como core-site.xml, hdfs-site.xml, capacity-scheduler.xml y archivos de entorno de Hadoop.

También existen configuraciones separadas para Spark, los JMX Exporters y el servidor frontal Apache HTTPD.

Diagrama de arquitectura



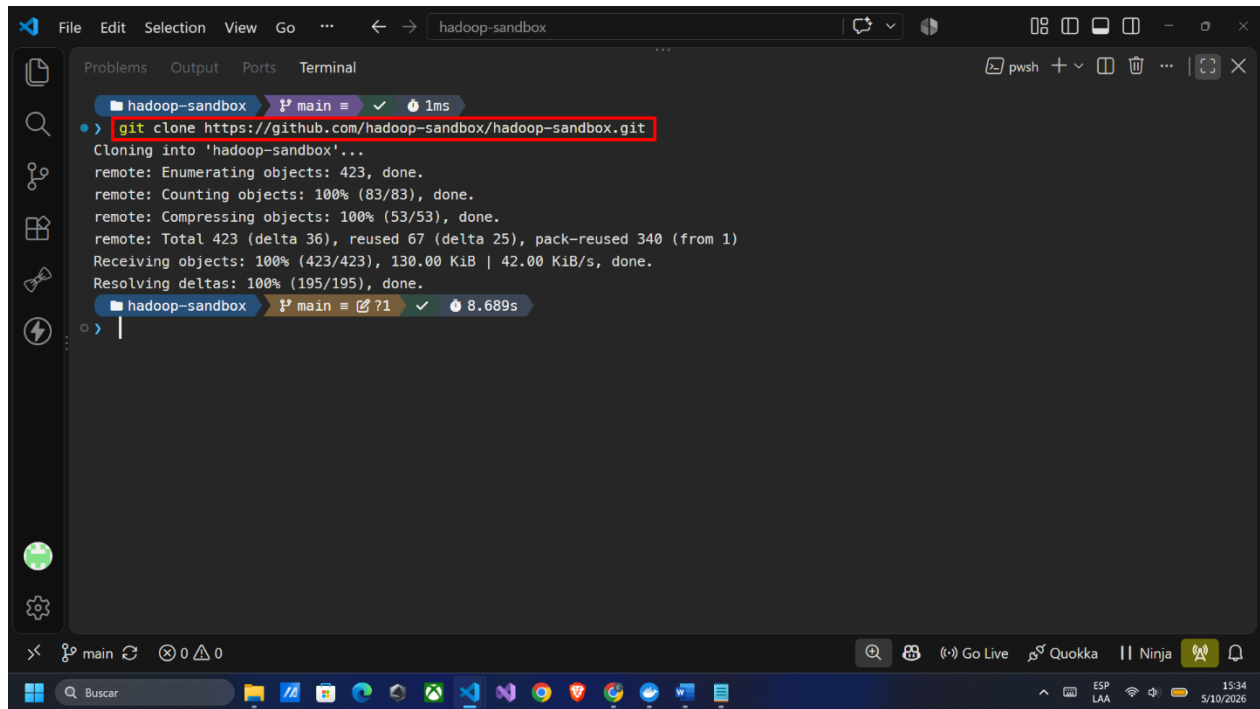
3. Implementación práctica

Para comprobar el funcionamiento del repositorio Hadoop Sandbox, se realizó el despliegue del clúster utilizando Docker Compose y posteriormente se verificó el funcionamiento de HDFS.

Paso 1. Clonar el repositorio

Se clonó el repositorio oficial desde GitHub mediante:

- git clone <https://github.com/hadoop-sandbox/hadoop-sandbox.git>



Paso 2. Ingresar al directorio

Una vez finalizada la clonación, se ingresó al directorio del proyecto:

- `cd hadoop-sandbox`

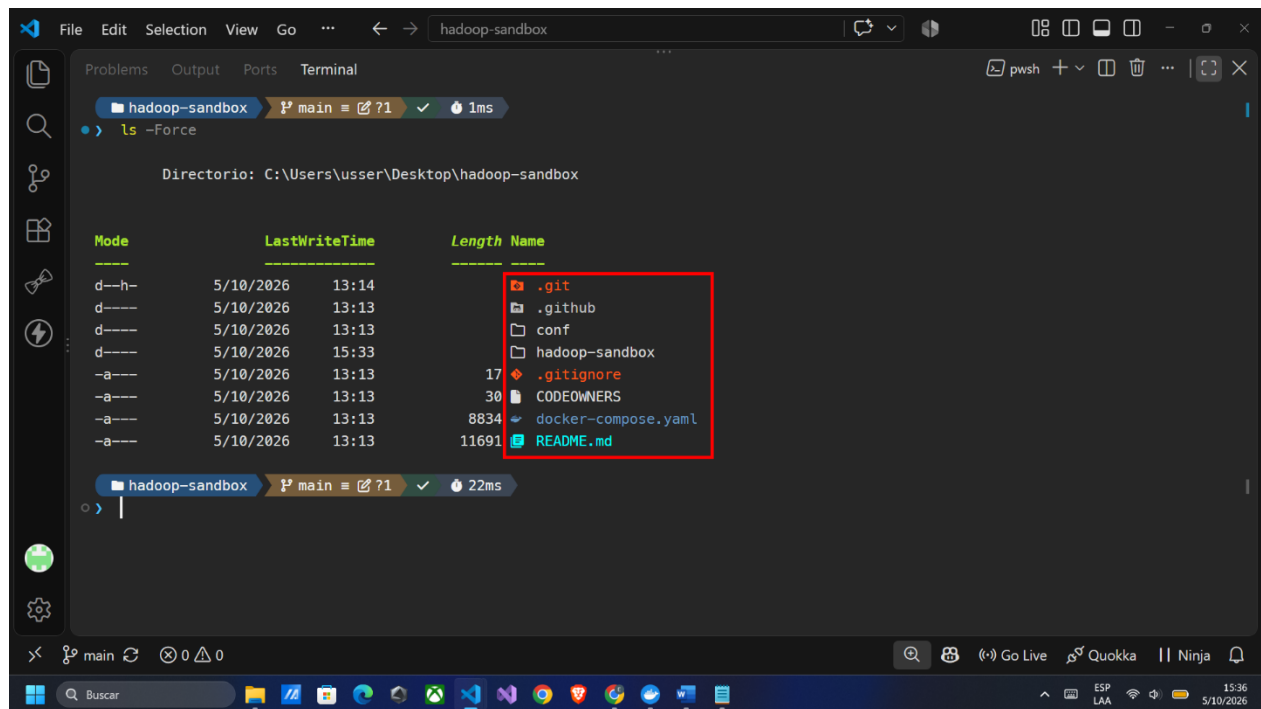
Paso 3. Identificar los archivos principales

Se listaron los archivos y directorios del repositorio mediante:

- `ls -Force`

Entre los elementos principales se identificaron:

- **docker-compose.yaml:** definición de los servicios y contenedores.
- **README.md:** documentación para instalación, configuración y uso.
- **conf/:** archivos de configuración de Hadoop, Spark, JMX Exporter y el servidor frontal.



Paso 4. Analizar Docker Compose

Se revisó el archivo principal de Docker Compose mediante:

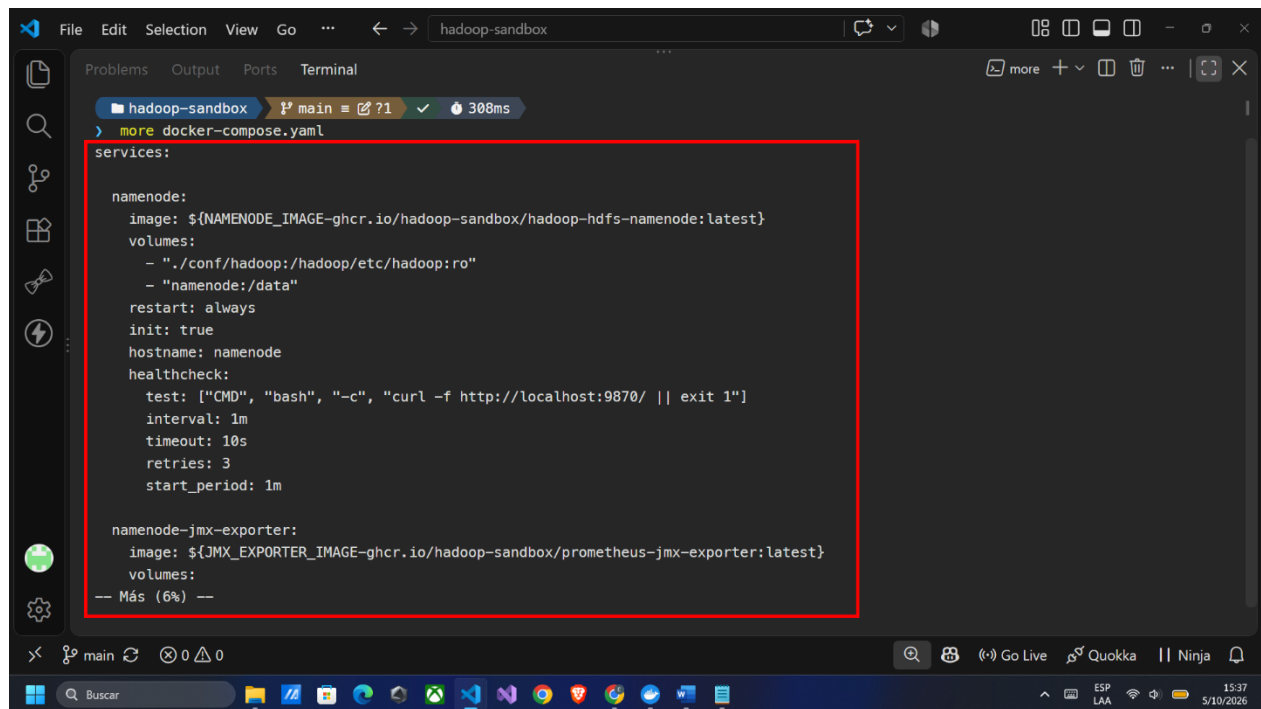
- `cat docker-compose.yaml`
- `more docker-compose.yaml`

En este archivo se encuentran definidos los diferentes servicios que conforman el clúster, entre ellos NameNode, DataNode, ResourceManager, NodeManager, JobHistoryServer, Spark History Server, el nodo cliente y los servicios de monitoreo.

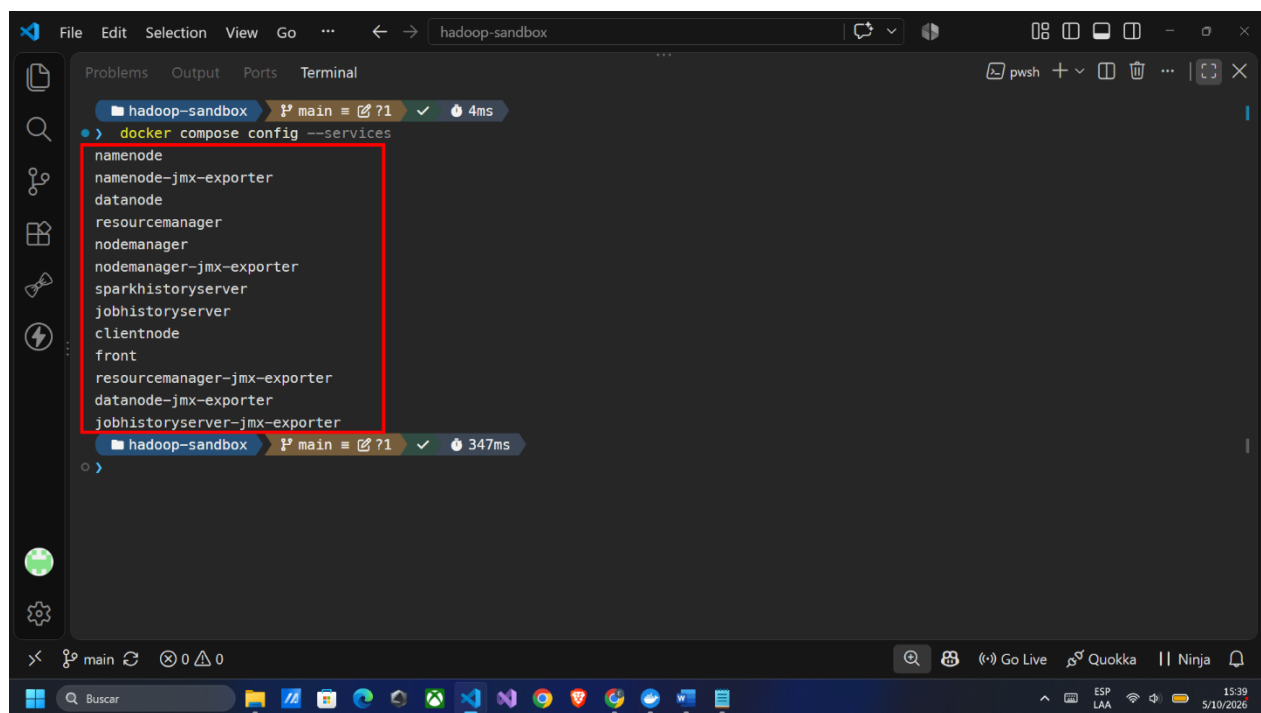
También se comprobó la cantidad de servicios definidos mediante:

- `docker compose config --services`

El despliegue contiene un total de 13 servicios.



```
File Edit Selection View Go ... hadoop-sandbox
Problems Output Ports Terminal
hadoop-sandbox main 71 308ms
> more docker-compose.yaml
services:
  namenode:
    image: ${NAMENODE_IMAGE-ghcr.io/hadoop-sandbox/hadoop-hdfs-namenode:latest}
    volumes:
      - ../conf/hadoop:/hadoop/etc/hadoop:ro
      - namenode:/data
    restart: always
    init: true
    hostname: namenode
    healthcheck:
      test: ["CMD", "bash", "-c", "curl -f http://localhost:9870/ || exit 1"]
      interval: 1m
      timeout: 10s
      retries: 3
      start_period: 1m
  namenode-jmx-exporter:
    image: ${JMX_EXPORTER_IMAGE-ghcr.io/hadoop-sandbox/prometheus-jmx-exporter:latest}
    volumes:
      - Más (6%)
-- Más (6%) --
main 0 0
```



```
File Edit Selection View Go ... hadoop-sandbox
Problems Output Ports Terminal
hadoop-sandbox main 71 4ms
> docker compose config --services
namenode
namenode-jmx-exporter
datanode
resourcemanager
nodemanager
nodemanager-jmx-exporter
sparkhistoryserver
jobhistoryserver
clientnode
front
resourcemanager-jmx-exporter
datanode-jmx-exporter
jobhistoryserver-jmx-exporter
hadoop-sandbox main 71 347ms
>
```

Paso 5. Ejecutar el proyecto

Posteriormente se inició el clúster en segundo plano:

- `docker compose up -d`

Paso 6. Verificar los contenedores

Se verificó el estado de los contenedores mediante:

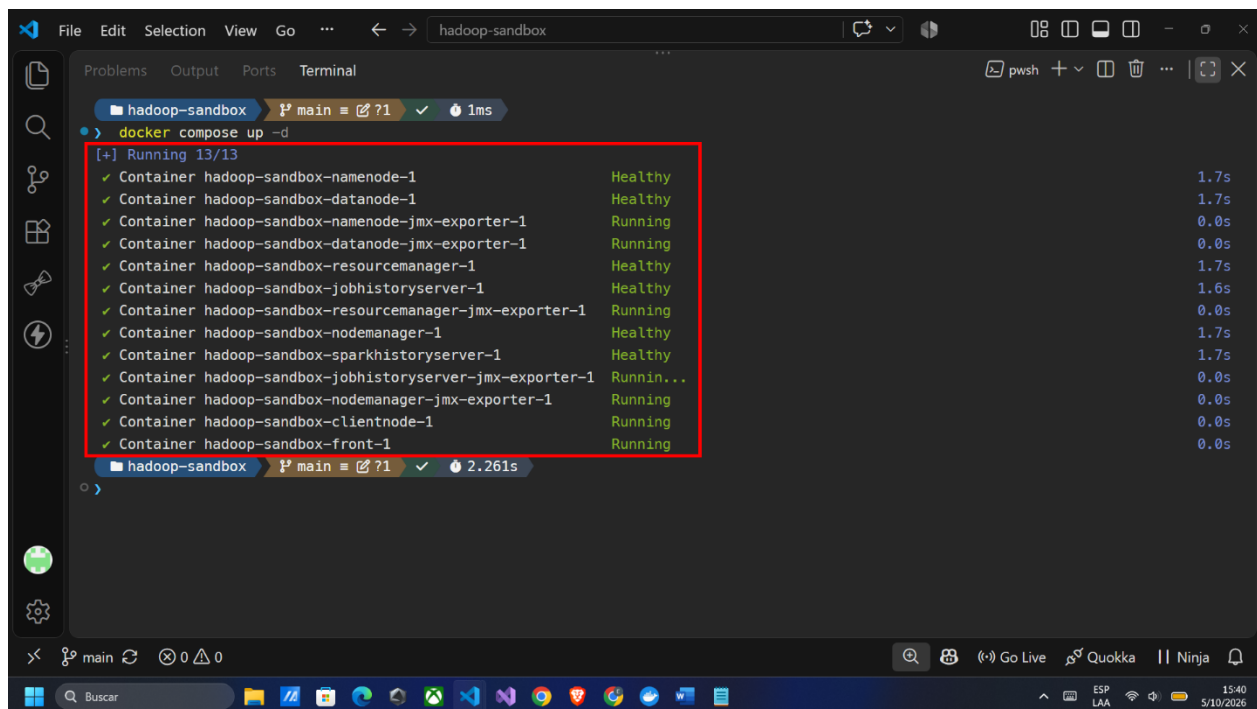
- `docker ps`

Los servicios principales aparecieron en ejecución y con sus verificaciones de salud correctas.

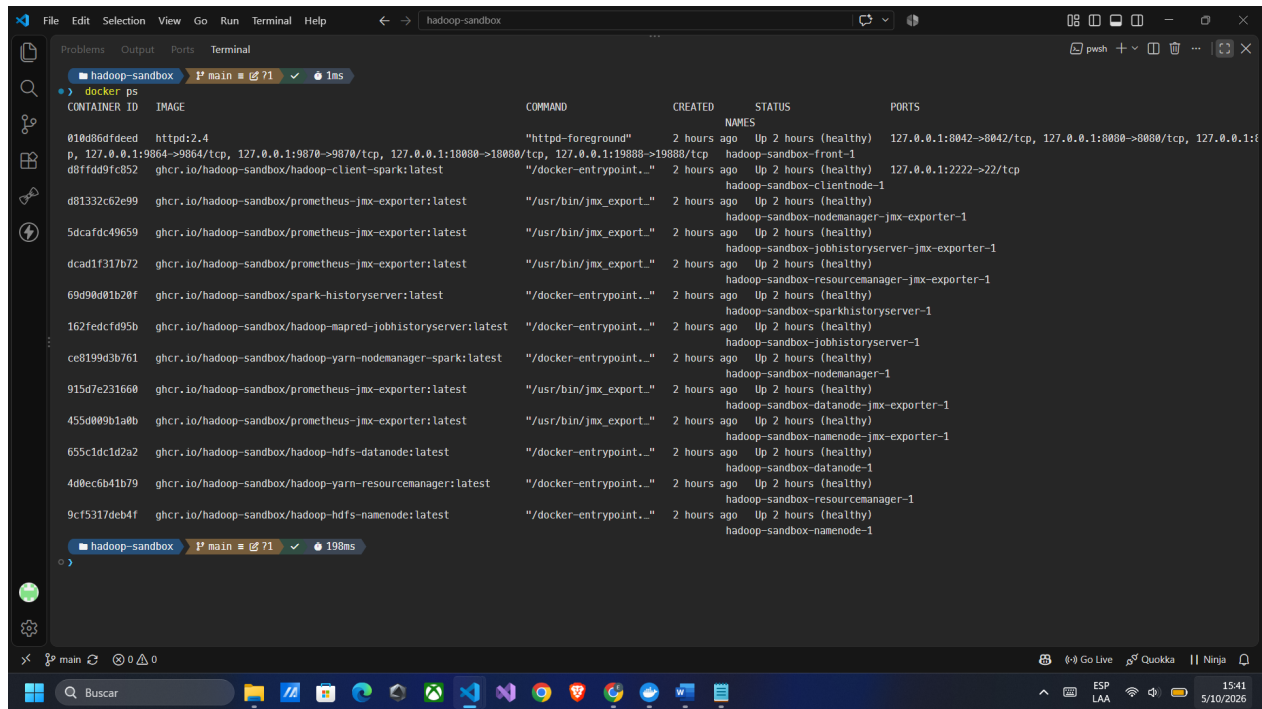
Adicionalmente, se comprobó el acceso a las interfaces web del clúster, principalmente:

- NameNode / HDFS: <http://localhost:9870>
- YARN ResourceManager: <http://localhost:8088>
- Página general: <http://localhost:8080>

Con estas comprobaciones se confirmó que el clúster Hadoop se encontraba operativo.



```
hadoop-sandbox main 71 1ms
[+] Running 13/13
✓ Container hadoop-sandbox-namenode-1 Healthy 1.7s
✓ Container hadoop-sandbox-datanode-1 Healthy 1.7s
✓ Container hadoop-sandbox-namenode-jmx-exporter-1 Running 0.0s
✓ Container hadoop-sandbox-datanode-jmx-exporter-1 Running 0.0s
✓ Container hadoop-sandbox-resourcemanager-1 Healthy 1.7s
✓ Container hadoop-sandbox-jobhistoryserver-1 Healthy 1.6s
✓ Container hadoop-sandbox-resourcemanager-jmx-exporter-1 Running 0.0s
✓ Container hadoop-sandbox-nodemanager-1 Healthy 1.7s
✓ Container hadoop-sandbox-sparkhistoryserver-1 Healthy 1.7s
✓ Container hadoop-sandbox-jobhistoryserver-jmx-exporter-1 Running... 0.0s
✓ Container hadoop-sandbox-nodemanager-jmx-exporter-1 Running 0.0s
✓ Container hadoop-sandbox-clientnode-1 Running 0.0s
✓ Container hadoop-sandbox-front-1 Running 0.0s
hadoop-sandbox main 71 2.261s
```



Verificación mediante interfaces web

Además de comprobar el estado de los contenedores desde la terminal, se verificó el funcionamiento del clúster accediendo a las diferentes interfaces web proporcionadas por Hadoop Sandbox. El repositorio expone interfaces para HDFS, YARN, MapReduce, Spark y los nodos del clúster.

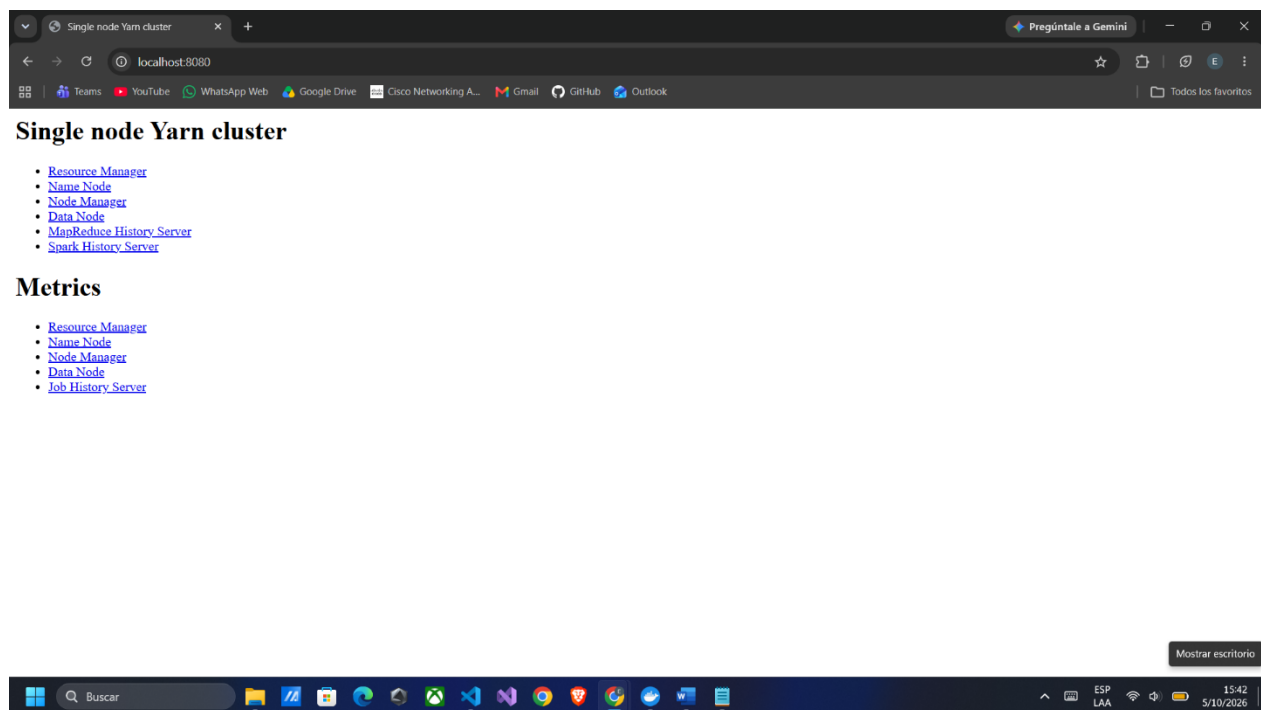
Las principales interfaces verificadas fueron:

Servicio	Dirección	Comprobación realizada
Página general	http://localhost:8080	Acceso general a los servicios del clúster
YARN ResourceManager	http://localhost:8088	Visualización del estado y recursos disponibles en YARN
HDFS NameNode	http://localhost:9870	Verificación del estado de HDFS y del NameNode
DataNode	http://localhost:9864	Comprobación del nodo encargado del almacenamiento
NodeManager	http://localhost:8042	Verificación del nodo de ejecución de YARN
MapReduce History Server	http://localhost:19888	Consulta de información e historial de trabajos MapReduce

Spark History Server	http://localhost:18080	Acceso al historial de trabajos ejecutados con Spark
WebHDFS	http://localhost:9870/explorer.html	Exploración del sistema de archivos HDFS desde una interfaz web

Al acceder correctamente a estas páginas se comprobó que los principales componentes del clúster se encontraban activos y accesibles desde el navegador.

En particular, mediante la interfaz del NameNode y WebHDFS fue posible comprobar visualmente el funcionamiento del sistema de almacenamiento HDFS, complementando la verificación realizada posteriormente mediante comandos.



Namenode information

Pregúntale a Gemini

localhost:9870/dfshealth.html#tab-overview

TeamsYouTubeWhatsApp WebGoogle DriveCisco Networking A...GmailGitHubOutlook

Todos los favoritos

Hadoop

Overview

Datanodes

Datanode Volume Failures

Snapshot

Startup Progress

Utilities

Overview 'namenode:8020' (✓active)

Started:	Mon Oct 05 13:20:10 -0400 2026
Version:	3.5.0, rUnknown
Compiled:	Sat Oct 03 00:31:00 -0400 2026 by root from Unknown
Cluster ID:	CID-e2907e3b-4076-4fe2-9042-d72e3477424f
Block Pool ID:	BP-1000514758-172.19.0.2-1791220802527

Summary

Security is off.
Safemode is off.

8 files and directories, 0 blocks (0 replicated blocks, 0 erasure coded block groups) = 8 total filesystem object(s).

Heap Memory used 98.6 MB of 150 MB Heap Memory. Max Heap Memory is 1.85 GB.

Non Heap Memory used 84.02 MB of 85.63 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:

1006.85 GB

Mostrar escritorio

Buscar

ESP LAA

15:42 5/10/2026

About the Cluster

Pregúntale a Gemini

localhost:8088/cluster

TeamsYouTubeWhatsApp WebGoogle DriveCisco Networking A...GmailGitHubOutlook

Todos los favoritos

hadoop

About the Cluster

Logged in as: dr.who

Cluster

About

Nodes

Node Labels

Applications

NEW

NEW SAVING

SUBMITTED

ACCEPTED

RUNNING

FINISHED

FAILED

KILLED

Scheduler

Tools

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Used Resources	Total Resources	Reserved Resources	Physical Mem Used %	Physical VCores Used %
0	0	0	0	0	<memory:0 B, vCores:0>	<memory:8 GB, vCores:8>	<memory:0 B, vCores:0>	43	0

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	Unhealthy Nodes	Rebooted Nodes	Shutdown Nodes
1	0	0	0	0	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation	Maximum Cluster Application Priority	Scheduler Busy %	RM Dispatcher EventQueue Size	Scheduler Dispatcher EventQueue Size
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:1024, vCores:1>	<memory:8192, vCores:4>	0	0	0	0

Cluster overview

Cluster ID:

1791220821103

ResourceManager state:

STARTED

ResourceManager HA state:

active

ResourceManager HA zookeeper connection state:

Could not find leader elector. Verify both HA and automatic failover are enabled.

ResourceManager RMStateStore:

org.apache.hadoop.yarn.server.resourcemanager.recovery.NullRMStateStore

ResourceManager started on:

Mon Oct 05 17:20:21 +0000 2026

ResourceManager version:

3.5.0 from Unknown by root source checksum 5f245a9fee3873666f512583c5df5d7 on 2026-10-03T04:36Z

Hadoop version:

3.5.0 from Unknown by root source checksum 64582ed0a62a169c39a19c734a8f33b5 on 2026-10-03T04:31Z

Paso 7. Prueba funcional

Para realizar una prueba directamente relacionada con la tecnología principal del proyecto, se utilizó **Hadoop Distributed File System (HDFS).**

Se accedió al nodo cliente del clúster mediante:

- `docker compose exec -u sandbox clientnode bash`

Desde este contenedor se realizaron las operaciones de creación de directorio, carga y consulta de archivos en HDFS.

4. Prueba funcional obligatoria – Hadoop/HDFS

La prueba tuvo como objetivo verificar que HDFS permite almacenar y consultar información correctamente dentro del clúster desplegado.

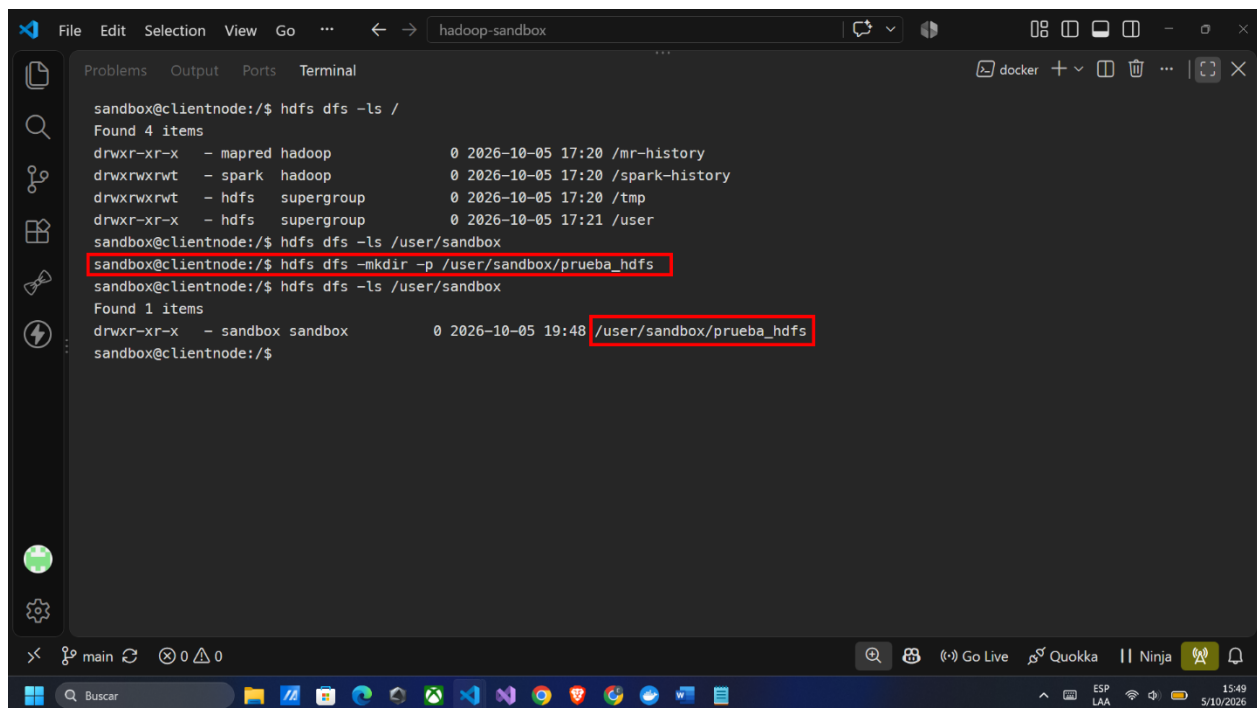
1. Creación de un directorio

Se creó un directorio denominado `prueba_hdfs` dentro del espacio del usuario `sandbox`:

- `hdfs dfs -mkdir -p /user/sandbox/prueba_hdfs`

Posteriormente se verificó su existencia:

- `hdfs dfs -ls /user/sandbox`



The screenshot shows a terminal window titled 'hadoop-sandbox' with the following commands and output:

```
sandbox@clientnode:/$ hdfs dfs -ls /
Found 4 items
drwxr-xr-x - mapred hadoop          0 2026-10-05 17:20 /mr-history
drwxrwxrwt - spark hadoop          0 2026-10-05 17:20 /spark-history
drwxrwxrwt - hdfs supergroup       0 2026-10-05 17:20 /tmp
drwxr-xr-x - hdfs supergroup       0 2026-10-05 17:21 /user
sandbox@clientnode:/$ hdfs dfs -ls /user/sandbox
sandbox@clientnode:/$ hdfs dfs -mkdir -p /user/sandbox/prueba_hdfs
sandbox@clientnode:/$ hdfs dfs -ls /user/sandbox
Found 1 items
drwxr-xr-x - sandbox sandbox      0 2026-10-05 19:48 /user/sandbox/prueba_hdfs
sandbox@clientnode:/$
```

The terminal window is part of a VS Code interface. The bottom status bar shows the file explorer with 'main' selected, 0 errors, 0 warnings, and 0 info messages. The system tray at the bottom right indicates the time is 15:49 on 5/10/2026, with language set to ESP/LAA.

2. Creación y carga de un archivo

Primero se creó un archivo de prueba en el sistema local del contenedor:

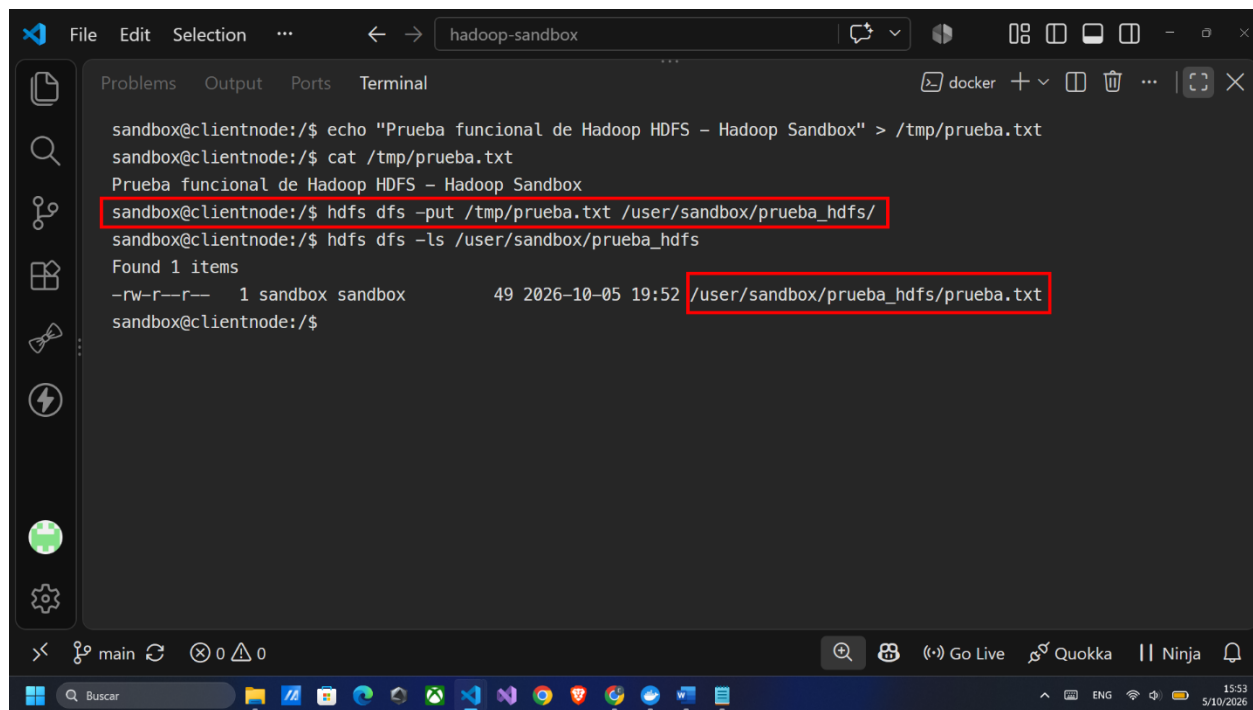
- `echo "Prueba funcional de Hadoop HDFS - Hadoop Sandbox" > /tmp/prueba.txt`

Luego se cargó el archivo en el directorio previamente creado en HDFS:

- `hdfs dfs -put /tmp/prueba.txt /user/sandbox/prueba_hdfs/`

Se verificó el archivo almacenado mediante:

- `hdfs dfs -ls /user/sandbox/prueba_hdfs`



The screenshot shows a terminal window titled 'hadoop-sandbox' with the following commands and output:

```
sandbox@clientnode:/$ echo "Prueba funcional de Hadoop HDFS - Hadoop Sandbox" > /tmp/prueba.txt
sandbox@clientnode:/$ cat /tmp/prueba.txt
Prueba funcional de Hadoop HDFS - Hadoop Sandbox
sandbox@clientnode:/$ hdfs dfs -put /tmp/prueba.txt /user/sandbox/prueba_hdfs/
sandbox@clientnode:/$ hdfs dfs -ls /user/sandbox/prueba_hdfs
Found 1 items
-rw-r--r-- 1 sandbox sandbox      49 2026-10-05 19:52 /user/sandbox/prueba_hdfs/prueba.txt
sandbox@clientnode:/$
```

The terminal output shows the successful upload of the file to HDFS. The file path `/user/sandbox/prueba_hdfs/prueba.txt` is highlighted in red in the original image.

3. Consulta del archivo almacenado

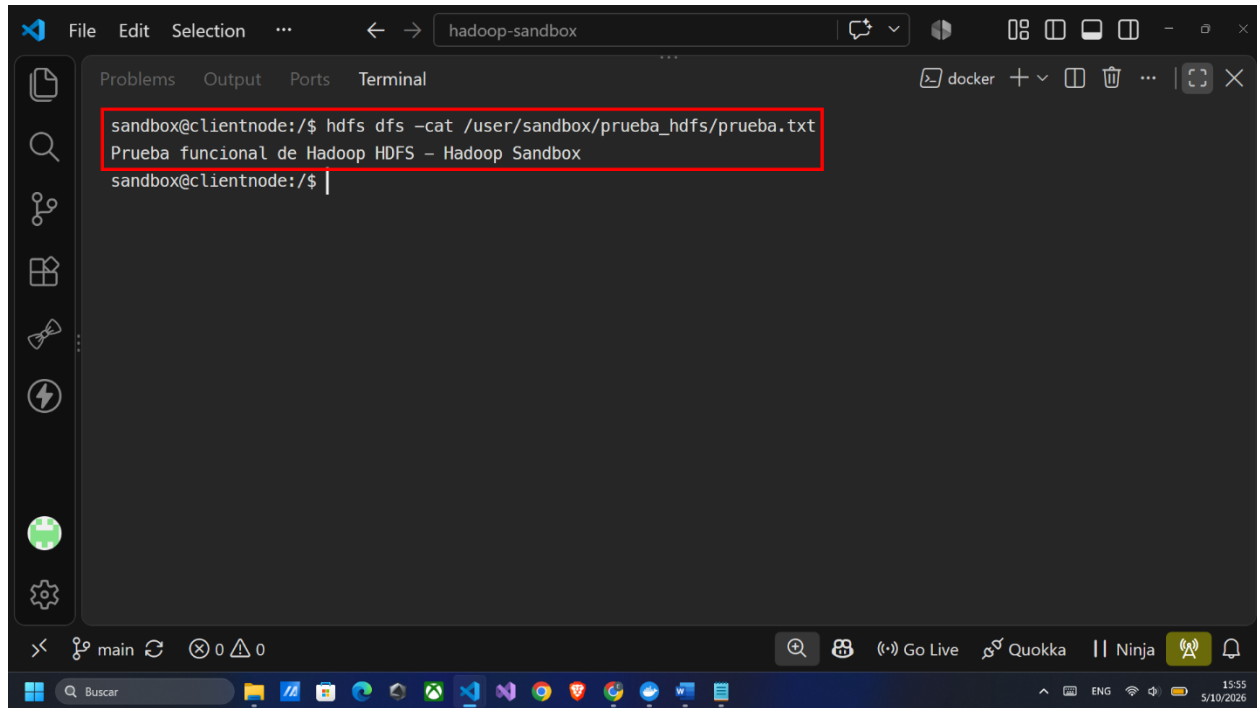
Finalmente, se consultó directamente el contenido del archivo utilizando:

- `hdfs dfs -cat /user/sandbox/prueba_hdfs/prueba.txt`

El resultado obtenido fue:

- Prueba funcional de Hadoop HDFS - Hadoop Sandbox

Esto confirmó que HDFS pudo localizar y recuperar correctamente el archivo almacenado.



The screenshot shows a terminal window titled 'hadoop-sandbox' within the Visual Studio Code editor. The terminal output is as follows:

```
sandbox@clientnode:/$ hdfs dfs -cat /user/sandbox/prueba_hdfs/prueba.txt
Prueba funcional de Hadoop HDFS - Hadoop Sandbox
sandbox@clientnode:/$
```

The command and its output are highlighted with a red rectangular box. The Visual Studio Code interface includes a sidebar with icons for Explorer, Search, Source Control, Run and Debug, and Extensions. The bottom status bar shows the current file is 'main', there are no errors or warnings, and the system clock indicates 15:55 on 5/10/2026.

4. Verificación del archivo desde la interfaz WebHDFS

Como comprobación adicional, se accedió a la interfaz web de HDFS desde el navegador mediante:

- <http://localhost:9870/explorer.html>

Esta interfaz permite explorar gráficamente los directorios y archivos almacenados en HDFS. El repositorio incluye esta dirección como interfaz WebHDFS.

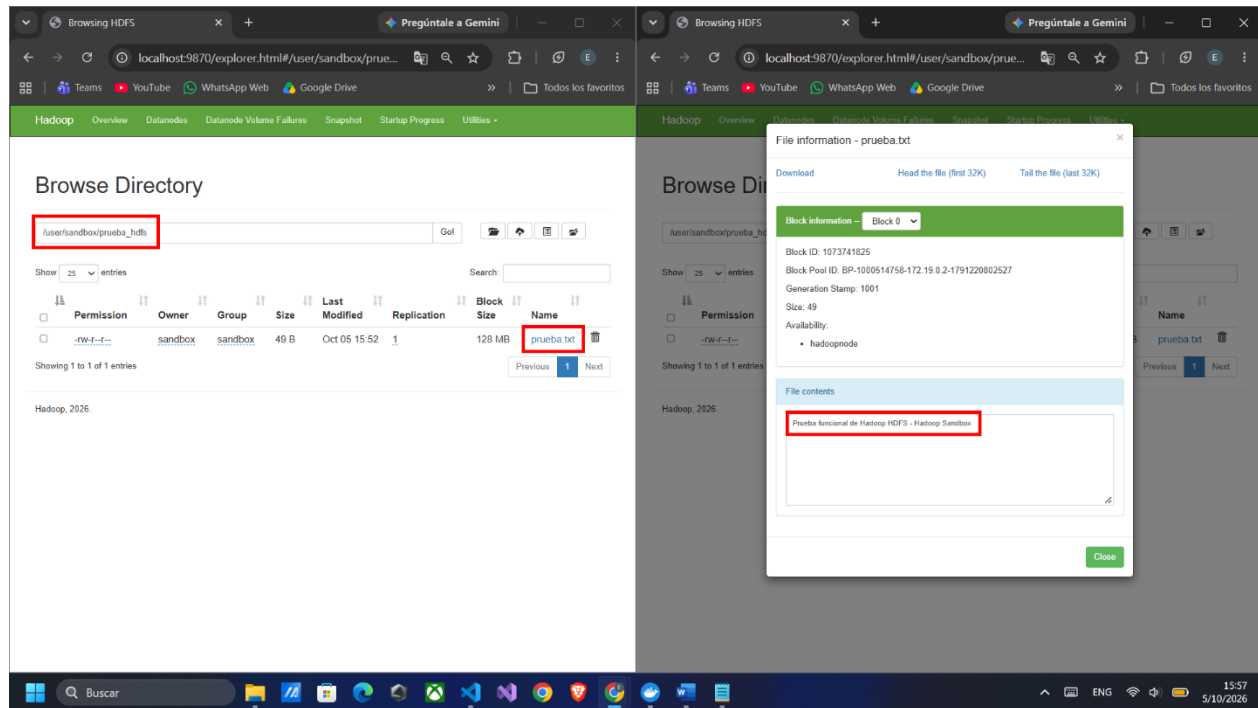
Dentro del explorador se navegó hasta la ruta:

- /user/sandbox/prueba_hdfs

En esta ubicación se verificó la existencia del archivo:

- prueba.txt

La visualización del archivo desde la interfaz web permitió confirmar que el archivo creado y cargado mediante comandos se encontraba correctamente almacenado en HDFS.



Resultado de la prueba

La prueba funcional fue exitosa, ya que se pudieron completar las tres operaciones requeridas:

Operación	Resultado
Creación de directorio en HDFS	Correcto
Carga de archivo en HDFS	Correcto
Consulta del archivo almacenado	Correcto

Por lo tanto, se verificó que el clúster Hadoop Sandbox fue desplegado correctamente y que su sistema de almacenamiento distribuido HDFS se encuentra funcional.

5. Comparación con docker-hadoop

El proyecto de Big Data Europe utiliza un despliegue más reducido compuesto principalmente por NameNode, DataNode, ResourceManager, NodeManager y HistoryServer, es decir, 5 servicios principales

en su docker-compose.yml. En cambio, Hadoop Sandbox incorpora servicios adicionales de Spark, monitoreo JMX, un nodo cliente y un frontal HTTP.

Característica	docker-hadoop	Hadoop Sandbox
Tecnología principal	Apache Hadoop	Apache Hadoop
Docker	Sí	Sí
Docker Compose	Sí	Sí
Número de contenedores	5 en el Compose principal	13
Almacenamiento distribuido	HDFS	HDFS
Procesamiento distribuido	YARN + MapReduce	YARN + MapReduce + Spark
Interfaces web	NameNode, DataNode, ResourceManager, NodeManager e HistoryServer	Incluye las anteriores y agrega Spark History Server, WebHDFS y página general
Persistencia	3 volúmenes principales	5 volúmenes
Complejidad de instalación	Baja-media; arquitectura más reducida	Media; más servicios, pero se inicia con docker compose up
Documentación	Documentación breve orientada al despliegue de Hadoop	Más detallada; incluye SSH, pruebas MapReduce/Spark, WebHDFS y configuración
Caso de uso	Crear rápidamente un clúster Hadoop en Docker	Entorno sandbox más completo para experimentar con Hadoop, YARN, HDFS, MapReduce y Spark

docker-hadoop utiliza Hadoop 3.2.1 en sus imágenes principales y ofrece interfaces para NameNode, DataNode, ResourceManager, NodeManager e History Server.

La diferencia principal es que docker-hadoop se concentra en un clúster Hadoop relativamente básico, mientras que Hadoop Sandbox amplía ese entorno con Spark, monitoreo JMX, acceso SSH y un servidor frontal para las interfaces web. No se trata de determinar cuál proyecto es mejor, sino de observar que ambos poseen objetivos y arquitecturas de distinta amplitud.

6. Conclusiones

La implementación del repositorio Hadoop Sandbox permitió comprender de forma práctica cómo puede desplegarse un entorno Big Data distribuido utilizando Docker y Docker Compose. Durante el desarrollo se identificaron los principales componentes de la arquitectura, como HDFS, YARN, MapReduce y Spark, además de los servicios de monitoreo y acceso incluidos en el proyecto.

El despliegue del clúster se realizó correctamente y se comprobó su funcionamiento tanto desde la terminal como desde las interfaces web proporcionadas por los distintos servicios. Esto permitió verificar que los contenedores se encontraban activos y que componentes como NameNode, ResourceManager y WebHDFS podían ser consultados desde el navegador.

La prueba funcional realizada sobre HDFS permitió crear un directorio, cargar un archivo y posteriormente consultar su contenido. Además, el archivo almacenado pudo verificarse desde la interfaz WebHDFS, demostrando que el sistema de almacenamiento distribuido se encontraba funcionando correctamente.

Finalmente, la comparación con Big Data Europe - docker-hadoop permitió identificar diferencias en la arquitectura y propósito de ambos proyectos. Aunque los dos utilizan Hadoop y Docker Compose, Hadoop Sandbox incorpora servicios adicionales como Spark, monitoreo mediante JMX Exporter, acceso por nodo cliente y una interfaz frontal, ofreciendo un entorno más amplio para realizar pruebas y experimentar con distintas tecnologías del ecosistema Hadoop.

En conclusión, la práctica permitió cumplir con el objetivo de encontrar, comprender, desplegar, probar y analizar un repositorio Big Data real basado en contenedores, fortaleciendo la comprensión de la arquitectura y funcionamiento de un clúster Hadoop.